OS Unit 4 Assignment

Name- Vikrant Acharyya
SRN- PES2UG24CS587
Section- 4J

Phase 1-

```
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
```

```
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2-

```
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

```
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
```

Phase 3-

```
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ echo "hello" > file1.txt
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ echo "world" > file2.txt
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ ./pes add file1.txt file2.txt
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.c
  untracked:  object.c
  untracked:  .gitignore
  untracked:  index.h
  untracked:  Makefile
  untracked:  tree.h
  untracked:  commit.c
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  index.c
  untracked:  commit.h
  untracked:  pes.c
  untracked:  object.h
  untracked:  README.md
  untracked:  test_tree
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  pes.h
```

```
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776418495 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776418504 6 file2.txt
```

Phase 4-

```
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ ./pes log
commit e17bc8d9c327436ef00ddbaadbd0fa88c37ad2337fb1f1153880f7e7ab4a8483
Author: PES User <pes@localhost>
Date:   1776419404

    Add farewell

commit 40c8ec94c7a6d817ce68c35c86a212398f47fd78806bad5a9d66c80f21dcfb6b
Author: PES User <pes@localhost>
Date:   1776419162

    Add world

commit 2f4b3505e081888e3f11a8898bb16835bea7b3e8e4df0a3bb2cb6fc1d31edaa2
Author: PES User <pes@localhost>
Date:   1776419138

    Initial commit
```

```
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/2f/4b3505e081888e3f11a8898bb16835bea7b3e8e4df0a3bb2cb6fc1d31edaa2
.pes/objects/40/c8ec94c7a6d817ce68c35c86a212398f47fd78806bad5a9d66c80f21dcfb6b
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e1/7bc8d9c327436ef00ddbaadbd0fa88c37ad2337fb1f1153880f7e7ab4a8483
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/refs/heads/main
```

```
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ cat .pes/refs/heads/main
e17bc8d9c327436ef00ddbaadbd0fa88c37ad2337fb1f1153880f7e7ab4a8483
vikky@vikky-Standard-PC-Q35-ICH9-2009:~/PES2UG24CS587-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Phase 5-

Q5.1 — How would you implement `pes checkout <branch>`?

ANS- Files that change in `.pes/`:

1. `.pes/HEAD` — updated to `ref: refs/heads/<branch>` (or a commit hash for detached HEAD).
2. `.pes/refs/heads/<branch>` — read (not written) to get the target commit hash.

What must happen to the working directory:

1. Read the current HEAD commit → get its root tree hash.
2. Read the target branch's commit → get its root tree hash.
3. Walk both trees recursively and compute the diff: which files were added, removed, or changed.
4. For each removed file: delete it from the working directory.
5. For each added or changed file: look up the blob hash from the target tree → read the blob from the object store → write its contents to the correct path, re-creating any missing directories.
6. Update the index so it reflects the new branch's tree (replace all index entries with the entries from the target tree, with freshly `stat()`-ed metadata).

What makes this complex:

- Conflicts with the working directory: if a file tracked by the *current* branch has been modified in the working directory but not staged, and that same file differs in the target branch, checkout must refuse — otherwise the user's uncommitted work would be silently overwritten.

Detecting this requires comparing the working file's mtime/size against the index (see Q5.2).
- Directory management: creating and removing directories as needed when a file moves in or out of a subdirectory; cleaning up empty directories left behind.
- Atomicity: a crash partway through leaves the working directory in a mixed state. Git works around this by validating the full diff before touching any files and by updating HEAD only after all file writes succeed.
- Untracked files: if an untracked file in the working directory has the same name as a file being added by the target branch, checkout must also refuse to avoid clobbering it.

Q5.2 — How would you detect a "dirty working directory" conflict before switching branches?

ANS- You need to identify files where three versions differ: the HEAD tree, the index, and the working directory file. Only using the index and the object store:

Step-by-step algorithm:

1. Read the current HEAD commit → get its root tree hash.
2. Walk the HEAD tree recursively to build a map: `path → blob_hash` for every file in the last commit.
3. Read the index to get the staged state: `path → { blob_hash, mtime_sec, size }`.
4. For each path in the index:
    - `stat()` the working file to get its current `mtime` and `size`.
    - If `mtime` or `size` differs from the stored index values, the file is modified in the working directory but not re-staged — it is *dirty*.
    - Now check: does this file's blob hash differ between the current branch's HEAD tree and the target branch's HEAD tree? If yes AND the file is dirty → conflict; refuse checkout.
5. For each path in the current HEAD tree that is absent from the index (i.e., deleted in the working directory): if the target branch's tree has a different version of that file → also a conflict.

The key insight is that `mtime`/`size` mismatches in the index are the fast-path detector for working-directory modifications — the same technique Git calls the "index invalidation" check — so you almost never need to re-hash files to detect dirtiness.

Q5.3 — What happens in "Detached HEAD" state, and how do you recover?

ANS- What detached HEAD means: Normally `.pes/HEAD` contains `ref: refs/heads/main` (a symbolic reference to a branch). In detached HEAD state it contains a raw commit hash directly, e.g. `a1b2c3d4....` There is no branch name involved.

Each `pes commit` still creates a valid commit object and still calls `head_update`, which writes the new hash directly back into `.pes/HEAD`. So you build a chain of real commit objects. However, because no branch file (under `.pes/refs/heads/`) points at your commits, they are reachable only through HEAD itself.

The moment you run `pes checkout <branch>` or `pes checkout <other-commit>`, HEAD is overwritten. Your chain of commits now has no reference pointing to it — it becomes unreachable and eligible for garbage collection.

How to recover those commits:

1. Before switching: note the commit hash printed by `pes log` (or read it directly from `.pes/HEAD` with `cat .pes/HEAD`).
2. Create a branch at that hash: `pes checkout -b recovery-branch <hash>` (if implemented), or manually write the hash into a new branch file:

```
echo "<commit-hash>" > .pes/refs/heads/recovery-branch
echo "ref: refs/heads/recovery-branch" > .pes/HEAD
```

3. If you have already switched away but GC has not run, the commit objects still exist on disk. You can recover by scanning `.pes/objects/` for commit objects (read each one, check its type), rebuild the chain manually, and create a branch pointing to the tip.

In real Git, `git reflog` records every value HEAD has ever held, making recovery trivial. PES-VCS could implement a simple append-only `.pes/logs/HEAD` file that logs each HEAD update with a timestamp for the same purpose.

<u>Phase 6</u>-

Q6.1 — Algorithm to find and delete unreachable objects

ANS- The mark-and-sweep algorithm:

Phase 1 — Mark (find all reachable objects):

Use a hash set (e.g. a hash table keyed on the 32-byte ObjectID) to track reachable hashes.

1. Collect all *roots*: read every file under `.pes/refs/heads/` and `.pes/HEAD` to get all branch-tip commit hashes.
2. For each root, do a depth-first traversal:
    - Mark the commit hash as reachable.
    - Read the commit → mark its tree hash as reachable.
    - Walk the tree recursively: for each entry, mark the blob or sub-tree hash, then recurse into sub-trees.
    - If the commit has a parent, recurse into the parent commit.
3. Continue until all reachable objects from all roots are marked.

Phase 2 — Sweep (delete unreachable objects):

1. Walk the entire `.pes/objects/` directory tree with `opendir/readdir`.
2. Reconstruct each ObjectID from the two-char shard name + filename.
3. If the hash is not in the reachable set → `unlink()` the file.
4. After unlinking, try `rmdir()` on the shard directory; it will silently fail if the directory is still non-empty.

Data structure: A hash set of `ObjectID` values — effectively an open-addressing or chaining hash table using the first N bytes of the SHA-256 as the bucket key. A sorted array with binary search also works if the reachable set fits in memory.

Estimation for 100,000 commits, 50 branches:

- Each commit references 1 tree object (root tree).
- A typical project might have ~10 tree objects per commit (root + a few subdirectory trees) and ~20 unique blob objects per commit on average (many shared across commits).
- Conservative estimate per commit: 1 commit + 10 trees + 20 blobs = ~31 objects.
- Total reachable objects: `100,000 × 31 ≈ 3.1 million objects`.
- In practice, deduplication means many blobs and trees are shared across commits, so you might visit each unique object only once — likely 500,000–1,000,000 actual object reads in a real repository.

Q6.2 — Race condition between GC and a concurrent commit

ANS-

Time →	GC thread	Commit thread
T1	Starts mark phase; reads all branch refs. Sees HEAD → C2.	
T2		Writes new blob B3 to object store.
	(B3 is not yet referenced by any commit; GC doesn't know about it yet.)	
T3	Mark phase complete. B3 is NOT in the reachable set because no commit or tree points to it yet.	
T4	Sweep phase: unlinks B3.	
T5		Builds tree T3 referencing B3. Writes tree T3 to object store.
T6		Writes commit C3 (tree=T3, parent=C2). Updates HEAD → C3.
T7		HEAD now points to C3, which points to T3, which points to B3. But B3 has been deleted by GC

The result is a dangling reference: the repository is now corrupt. Reading commit C3's tree will fail because blob B3 no longer exists on disk.

How Git avoids this:

1. Grace period / timestamp guard: Git's GC (`git gc`) skips any object file whose filesystem `mtime` is less than 2 weeks old (configurable via `gc.pruneExpire`). A freshly written object will have a recent mtime and will be spared even if it has no references yet. The 2-week window is much longer than any commit operation takes.
2. `gc.lock` file: `git gc --auto` creates a lock file (`.git/gc.pid`) before starting, so two GC processes cannot run simultaneously.

3. Write ordering for commits: Git always writes objects *before* updating the reference that makes them reachable. So a partial commit leaves orphaned objects (harmless — GC will clean them up after the grace period) but never leaves *missing* objects pointed to by live references.
4. Loose-object pack files: when Git packs objects it writes the pack file to a temp location and renames it atomically; the old loose objects are only deleted after the pack is fully verified.

The combination of the grace-period heuristic and careful write ordering is sufficient in practice, no locks are needed between GC and normal commit operations.